sedfit

User Manual

Photometry in, redshifts out

Version 0.1.0

Contents

1 What this does

`sedfit` takes photometry you already have for a set of galaxies and fits each one's spectral energy distribution to estimate its redshift.

You give it three things:

1. a **sample catalog** — a spreadsheet listing your galaxies, one row each;
2. a **campaign config** — a settings file saying which photometry files to expect and how to combine them;
3. a **fit config** — a settings file saying how to fit.

It then does three things:

1. **Builds** one combined photometry table per galaxy, putting every instrument on a common flux scale.
2. **Fits** that table with one of two engines and estimates a redshift with uncertainties.
3. **Records** everything — every input file is hashed, every setting is saved next to the results, and every run is logged in one central file.

Fluxes are in microjansky on the AB system throughout. Everything for one galaxy lands under that galaxy's own directory.

2 Installing

These instructions assume the machine has nothing set up yet. Commands are the same on all three operating systems unless stated otherwise.

2.1 Get Python

`sedfit` needs **Python 3.11 or newer** plus a scientific stack (NumPy, SciPy, Astropy, Pandas, Matplotlib).

Use **Miniconda**. Conda installs Python and the scientific libraries as pre-built binaries, which avoids the compiler toolchain some of these packages otherwise need, and it keeps each project's libraries separate so one project cannot break another.

Windows

1. Go to `https://www.anaconda.com/download/success` and download the **Miniconda Windows 64-bit installer** (`.exe`).
2. Run the installer and accept the defaults.
3. From the Start menu, open **Anaconda Prompt**. Every command in this manual is typed there.

macOS

1. Download the **Miniconda macOS installer** for your chip (Apple silicon = `arm64`, Intel = `x86_64`).
2. Run it, then open **Terminal**.

Linux

1. Download the Miniconda Linux `x86_64` installer script.
2. Run `bash Miniconda3-latest-Linux-x86_64.sh`, then open a new terminal.

Check that conda is available:

```
conda --version
```

You should see something like `conda 24.9.2`.

2.2 Get the code

The source lives at `https://github.com/chrishopp12/sed-fitting`.

With git. On Windows, install Git for Windows from `https://git-scm.com/download/win` first; macOS and Linux usually have git already. Then:

```
git clone https://github.com/chrishopp12/sed-fitting.git
```

This creates a folder named `sed-fitting`. Move into it:

```
cd sed-fitting
```

Without git. Open the page in a browser, click the green **Code** button, choose **Download ZIP**, and unzip it. You get a folder named `sed-fitting-main`. Move into it:

```
cd sed-fitting-main
```

You are in the right folder if it contains a file named `pyproject.toml`.

The download is about 60 MB

Most of that is the packaged template libraries — the galaxy spectra the fit compares your photometry against. They ship with the code so a fit is reproducible without downloading anything else.

2.3 Create an environment and install

Conda route (recommended, all platforms).

```
conda create -n sedfit python=3.12
```

Answer `y` when it asks. Then activate it:

```
conda activate sedfit
```

Your prompt now starts with `(sedfit)`. Install the package:

```
pip install -e .
```

That reads `pyproject.toml` and pulls in every dependency.

If a dependency fails to build, install the scientific stack from conda-forge first and then install `sedfit` without re-resolving:

```
conda install -c conda-forge numpy scipy pandas astropy matplotlib
```

```
pip install -e .  --no-deps
```

pip / venv route (no conda). Only use this if Python 3.11+ is already installed and on your PATH.

Windows:

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

macOS / Linux:

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

Then, on any platform:

```
pip install -e .
```

2.4 What "editable" means

`pip install -e .` installs the package **in place**: Python runs the source files in the folder you downloaded, rather than a copy hidden in the environment. If you `git pull` a newer version, the change takes effect immediately with no reinstall.

2.5 Optional extras

The base install runs everything in this manual. Two extras add alternative fitting engines:

```
pip install -e ".[eazy]"    # the official eazy-py engine
```

```
pip install -e ".[prospector]"    # Prospector + FSPS
```

Prospector on Windows

Prospector needs FSPS, which compiles from Fortran source and is difficult to build on Windows. The base install and both eazy engines run on any platform; if you need Prospector, use Linux or macOS.

Prospector also needs a data checkout and an environment variable:

```
git clone https://github.com/cconroy20/fsps ~/fsps
```

```
export SPS_HOME=~/fsps
```

Ready-made conda environments for both spectral libraries live in `envs/`:

```
conda env create -f envs/sedfit_miles.yml
```

2.6 Verify the install

```
sedfit --help
```

You should see a list of seven verbs: `roster`, `build`, `fit`, `batch`, `run`, `plot`, `manifest`. Then run the test suite:

```
python -m pytest tests/
```

Everything should pass. Tests for an engine you did not install skip themselves rather than failing.

3 Before you start

`sedfit` does not download photometry. It expects the photometry files to be on disk already, laid out one directory per galaxy.

3.1 The directory layout it expects

```
data_root/
    gal_0001/
        Photometry/
            gal_0001_catalog.csv
            gal_0001_measured.csv
            SPHEREx/
                table_photometry.csv
    gal_0002/
        Photometry/
            ...
```

You choose where `data_root` is and what the galaxy directories are called; the campaign config points at them. Names like `Photometry/` are conventions the campaign config declares, not rules baked into the code.

3.2 What a photometry file must contain

Every source CSV needs four columns:

Column	Meaning
`band`	the filter name, e.g. `Legacy_g`
`flux_uJy`	flux in microjansky
`flux_err_uJy`	its uncertainty
`source`	where the measurement came from

The `source` column matters more than it looks. One file can hold measurements from several archives, and `sedfit` tells them apart by the start of this string — `Legacy_DR9`, `SDSS_DR17_cModel`, and so on. It is how the same band can come from two different places in one file.

3.3 The three files you write

1. The sample catalog (a CSV, one row per galaxy). This is the file you edit as the sample grows.

```
name,ra_deg,dec_deg,dir,z_ref_kind,z_ref
gal_0001,217.51,56.99,gal_0001,spec,0.1043
gal_0002,217.62,57.02,gal_0002,spec,0.1058
gal_0003,217.44,56.95,gal_0003,reference,
```

Column	Meaning
`name`	the galaxy's name; must be unique
`ra_deg`, `dec_deg`	position in degrees
`dir`	its directory under `data_root` (optional; defaults to the name)
`z_ref_kind`	`spec`, `phot`, or `reference`
`z_ref`	the known redshift; **leave blank** for `reference`
`label`	the filename stem (optional; defaults from the name)

`z_ref_kind` says where the comparison redshift comes from. Use `spec` or `phot` when the galaxy has its own measured value, and `reference` when it should adopt the sample-wide redshift set in the campaign config. Columns `sedfit` does not recognize are ignored and reported, so you can keep your own notes in the same file.

2. The campaign config (`campaign.json`). Everything shared across galaxies. A minimal one:

```
{
    "schema_version": 1,
    "sample": "my_sample",
    "data_root": "..",
    "position_authority": "Legacy DR9 positions",
    "sources": {
        "legacy": {
            "path": "Photometry/{label}_catalog.csv",
            "bands": ["Legacy_g", "Legacy_r", "Legacy_z"],
            "kind": "catalog",
            "provider": "legacy"
        }
    },
    "recipes": {
        "plain": {
            "reference": "none",
            "sources": [{"source": "legacy", "role": "float"}],
            "spherex": null
        }
    }
}
```

`data_root` is relative to the campaign file. A *source* is one photometry file and the bands you expect from it; `{label}` is replaced with each galaxy's filename stem. A *recipe* is one way to combine those sources into a table — you can define several and run them all.

3. **The fit config** (e.g. `quick.json`). How to fit:

```
{
    "schema_version":  2,
    "backend":  "eazy",
    "name":  "quick_v1",
    "err_floor":  0.05,
    "eazy":  {
        "engine":  "quick",
        "z_min":  0.0,
        "z_max":  1.0,
        "z_step":  0.001,
        "templates":  "brown14_vac_cosmos160"
    }
}
```

You must name a template set

There is no default. The template set is the single biggest influence on the redshift you get out, so the code makes you choose it deliberately. Name any of the sets shipped in `sedfit/data/templates/` — `brown14_vac_cosmos160` is the recommended one — or give a path to your own directory of spectra.

4 The verbs

Verb	**What it does**
`roster`	Reads your sample catalog and campaign config, checks what is actually on disk, and writes `roster.json`
`build`	Combines each galaxy's photometry into one fit-ready table
`fit`	Fits one galaxy with one recipe
`batch`	Fits every galaxy, in parallel, and writes a report
`run`	Does `roster`, `build` and `batch` in one command
`plot`	Redraws the figures for a run you already have
`manifest`	Summarizes every run recorded so far

The normal working order is `roster` → `build` → `batch`. The `run` verb chains those three, and is what you want most of the time.

Every verb explains itself:

```
sedfit --help     sedfit build --help
```

5 Your first run

Put your sample catalog and campaign config next to each other, then:

Step 1 — check what the code can see. This writes nothing:

```
sedfit roster --catalog sample.csv --campaign campaign.json --dry-run
```

You get a line per galaxy-and-source that did not work out:

```
sample.csv:  3 rows; using name, ra_deg, dec_deg, z_ref_kind, z_ref, dir
3 catalog rows -> 3 targets
    1 dropped, 8 kept
    dropped  gal_0003/legacy:  Photometry/gal_0003_catalog.csv:  absent
dry run:  nothing written
```

Read that carefully before going on — it is telling you which files it found and which it did not.

Step 2 — write the roster. Drop `--dry-run`:

```
sedfit roster --catalog sample.csv --campaign campaign.json
```

This writes `roster.json` next to the campaign file, plus `roster.json.report.csv`, which has one row per galaxy-and-source with the reason it was kept, partially kept, or dropped.

Never edit roster.json by hand

It is generated. If something in it is wrong, fix the sample catalog or the campaign config and run `sedfit roster` again — it takes seconds. Hand edits are silently overwritten the next time anyone regenerates it.

Step 3 — build and fit everything:

```
sedfit batch --roster roster.json --config quick.json --build
```

You get a line per job as it finishes:

```
6 jobs (eazy), 3 worker(s); 0 pair(s) not applicable
[1/6] ok            gal_0001/plain z50=0.1043
[2/6] ok            gal_0002/plain z50=0.1061
...
6 ok, 0 failed, 0 skipped -> batch_report.csv
```

Or do all three steps at once:

```
sedfit run --catalog sample.csv --campaign campaign.json \
        --config quick.json
```

6 Common tasks

6.1 A note on long commands

The backslash at the end of a line means "the command continues on the next line." That works in Anaconda Prompt on Windows and in any macOS or Linux terminal. You can always type the whole thing on one line instead.

6.2 Add galaxies to the sample

Add rows to your sample catalog, then regenerate and re-run:

```
sedfit roster --catalog sample.csv --campaign campaign.json
sedfit batch --roster roster.json --config quick.json --build
```

Galaxies that were already fitted are skipped automatically, so only the new ones cost time. This is the everyday loop.

6.3 Fit one galaxy

```
sedfit fit --roster roster.json --target gal_0001 \
        --recipe plain --config quick.json
```

6.4 Check a command before running it

`--dry-run` validates everything and writes nothing. Use it whenever you have changed a config:

```
sedfit fit --roster roster.json --target gal_0001 \
        --recipe plain --config quick.json --dry-run
```

6.5 Build tables without fitting

```
sedfit build --roster roster.json
```

Add `--target` or `--recipe` to narrow it down.

6.6 Run only some galaxies

Put the names in a CSV with a `name` column — your sample catalog works directly — and pass it:

```
sedfit batch --roster roster.json --config quick.json \
        --targets subset.csv
```

Or a single one with `--target gal_0001`.

6.7 Run only one recipe

```
sedfit batch --roster roster.json --config quick.json --recipe plain
```

6.8 Correct for Milky Way dust

```
sedfit batch --roster roster.json --config quick.json \
        --build --deredden
```

This looks up the extinction at each galaxy's position and writes a separate `_dered` table, leaving the uncorrected one alone.

Use --build with --deredden

`--deredden` tells the fit to look for the dereddened table. If you do not also pass `--build`, that table may not exist and the fit will tell you so.

6.9 Redo a fit that already exists

By default a batch skips galaxies already fitted with the same settings. To force it:

```
sedfit batch --roster roster.json --config quick.json --no-resume
```

If it then complains that the existing run came from different software, add `--force`.

6.10 Use more or fewer processors

```
sedfit batch --roster roster.json --config quick.json --workers 4
```

`--workers 1` runs everything in one process, which makes error messages much easier to read. Use it when debugging.

6.11 Redraw figures

```
sedfit plot --run-dir gal_0001/SED/plain/eazy/64f456f8835f-quick_v1
```

6.12 See everything that has been run

```
sedfit manifest --roster roster.json
```

```
792 rows, 792 current runs, 0 superseded
    64f456f8835f  ok      gal_0001/SED/plain/eazy/...  z50=0.1043
```

7 Every flag, explained

`sedfit <verb> --help` always prints the current list. This section explains what the less obvious ones mean.

7.1 Shared by `fit`, `batch` and `run`

Flag	**Meaning**
`--force`	Replace an existing run even though it was produced by different package versions. Without this, the code refuses, because the old and new results would not be comparable.
`--allow-registry-drift`	Record a warning instead of stopping when a filter curve has changed since the table was built.
`--deredden`	Fit the Milky-Way-corrected (`_dered`) table.
`--no-plots`	Skip the figures. Faster for large sweeps.

7.2 `roster`

Flag	**Meaning**
`--catalog`	Your sample catalog CSV. Required.
`--campaign`	Your campaign config JSON. Required.
`--out`	Where to write the roster. Defaults to `roster.json` beside the campaign file.
`--dry-run`	Report what would be declared; write nothing.

7.3 `build`

Flag	**Meaning**
`--roster`	The roster JSON. Required.
`--target`	Build one galaxy; omit for all of them.
`--recipe`	Build one recipe; omit for every applicable one.
`--dry-run`	Validate every pair; write nothing.
`--deredden`	Also write the Milky-Way-corrected table.
`--mw-ebv`	Use one E(B–V) for every galaxy instead of looking each one up. Needs `--deredden`.

7.4 `fit`

Flag	**Meaning**
`--target`	Which galaxy.
`--recipe`	Which recipe's built table to fit.
`--config`	The fit config JSON.
`--label`	A suffix for the output directory name. Defaults to the `name` in the fit config.
`--jobs`	A JSON list of {target, recipe, config} objects to run in order, instead of naming one.
`--phot-csv`	Fit this photometry file instead of the built one. For one-off tests.
`--dry-run`	Resolve and validate; write nothing.

7.5 `batch` and `run`

Flag	**Meaning**
`--config`	The fit config applied to every galaxy. Required.
`--targets`	A CSV whose `name` column picks the galaxies.
`--target`	A single galaxy, instead of `--targets`.
`--recipe`	One recipe; omit for every applicable one.
`--workers`	How many processes. Omit to choose automatically.
`--build`	Rebuild each table before fitting it.
`--no-resume`	Re-run galaxies already finished.
`--report`	Where to write the report CSV.
`--log-dir`	Where to write the per-galaxy logs.

Flag	Meaning
`--stop-after-failures`	Give up after this many failures. `0` disables the limit.

`run` additionally takes `--catalog`, `--campaign` and `--out`, which behave exactly as they do for `roster`.

8 What comes out

8.1 The directory tree

```
gal_0001/
    Photometry/
        gal_0001_catalog.csv    (yours, untouched)
        gal_0001_sed_plain.csv  the combined table
        gal_0001_sed_plain.provenance.json
    SED/
        plain/                  one folder per recipe
            eazy/                 one folder per engine
                64f456f8835f-quick_v1/  one run
                    config.json
                    phot.csv
                    manifest.json
                    summary.csv
                    plots/
```

The long name on the run folder is not random. It is a fingerprint of the settings, the photometry and the filter curves. Fit the same galaxy the same way twice and you get the same folder — it is replaced, not duplicated. Change anything that matters and you get a new folder, so results from different settings can never be confused.

8.2 What each file is

File	What it holds
`_sed_<recipe>.csv`	The combined photometry the fit used
`.provenance.json`	Every input file, hashed, and every choice made
`config.json`	The complete settings, with defaults filled in
`phot.csv`	An exact copy of the photometry this run fitted
`manifest.json`	This run's result line
`summary.csv`	Redshifts and chi-square
`arrays.npz`	The redshift grid and the probability curve
`plots/`	The figures
`batch_report.csv`	One row per galaxy from a batch
`runs.jsonl`	Every run ever, appended

8.3 The numbers you probably want

`batch_report.csv` is the quickest place to look:

Column	**Meaning**
`target, recipe`	which galaxy, which recipe
`status`	`ok`, `failed`, or `skipped`
`zred_p50`	the redshift estimate (the median)
`zred_p16, zred_p84`	the 1-sigma range
`n_bands_fit`	how many bands actually entered the fit
`n_bands_table`	how many were available
`error`	why it failed, when it did

If `n_bands_fit` is much smaller than `n_bands_table`, bands were dropped — usually for low signal-to-noise. The run's `manifest.json` lists exactly which ones and why.

9 Reading the figures

Each run writes two figures into `plots/`.

The SED figure shows your photometry as points, with error bars, against the best-fitting template. Points are colored by instrument. If the template misses a group of points badly, that instrument may be on the wrong flux scale — check the recipe.

The redshift-scan figure shows how well each redshift fits. A good fit shows one clear, narrow minimum. Two things to watch for:

- **Several comparable minima** — the fit cannot decide, and the quoted uncertainty understates the real one.
- **The minimum at the edge of the range** — the true redshift is probably outside your `z_min`–`z_max` window. The code warns about this in the log too. Widen the range and re-run.

10 When something goes wrong

Error messages name the file and say what to do. The common ones:

`roster not found: ...`
The path after `--roster` is wrong, or you have not run `sedfit roster` yet.

`unknown target 'x'; the roster declares: ...`
A typo, or the galaxy was dropped during roster generation. Check `roster.json.report.csv` for why.

`... has not been built; run: sedfit build ...`
Exactly what it says — the message includes the command to run.

`templates=... is not a directory, a .param file, or a packaged set`
The message lists the available set names. Check the spelling in your fit config.

`only N fittable bands after inclusion and gates (min_valid_bands=5)`
Too few bands survived. Either the photometry is thin for that galaxy, or the signal-to-noise cut removed too much. Lower `min_valid_bands` in the fit config if that is scientifically appropriate.

`chi2 minimum on the grid edge; widen the grid`
Increase `z_max` or decrease `z_min`.

`existing run was produced by different machinery`
You reinstalled or upgraded something since that run. Add `--force` to replace it.

`no declared band` in the roster report
The file exists but holds none of the bands the campaign config expects from it. Usually the `provider` or the band names are wrong.

A command seems to hang.
The first step of every verb opens every photometry file the roster declares. On a network drive or a cloud-synced folder, that can take minutes the first time. It is faster afterwards.

An unreadable error during a batch.
Re-run the failing galaxy on its own with `--workers 1`:

```
sedfit fit --roster roster.json --target gal_0007 \
        --recipe plain --config quick.json
```

Per-galaxy logs from a batch are in `batch_logs/`.

11 Getting more out of it

11.1 Several recipes at once

A recipe is one way of combining your photometry. Define several in the campaign config and `batch` runs them all, giving each its own folder. Comparing recipes is how you test whether a result depends on how the instruments were combined.

11.2 Comparing template sets

Copy your fit config, change `templates` and `name`, and run both. Because the run folder is named after the settings, the two results sit side by side without overwriting each other. The shipped sets are:

Set	**What it is**
`brown14_vac_cosmos160`	The recommended set: Brown+14 galaxies plus the COSMOS templates
`brown14`	The Brown+14 atlas as published
`brown14_vac`	Brown+14, air-to-vacuum corrected
`ssp_miles`	FSPS simple stellar populations, MILES library
`ssp_c3k_a`	The same on the C3K library

11.3 The central log

Every finished run appends one line to `runs.jsonl` under your data root. It is one JSON object per line, so it loads directly into pandas:

```
import pandas as pd
runs = pd.read_json("sed_fitting/runs.jsonl", lines=True)
```

That is the easiest way to compare many runs at once.

11.4 Where to read next

`docs/technical_manual.md` in the repository explains how the code works internally — what each module does, how the flux scaling is computed, and what every output field means.